Supplementary Material: Derivations, Algorithm Specification, and Per-Instance Results for Tri-Level Hybrid DDQN-ALNS

Huynh Nhat Huy Thi-Linh Ho Nguyen Nhat Huy Nguyen Thi Bao Tran

November 9, 2026

1 Penalty Bound for Lexicographic Set Partitioning

Definition 1 (Lexicographic Set Partitioning over Finite Route Pool)**.** *Let $\mathcal{P} = \{R_1, \ldots, R_m\}$ be the finite pool of feasible routes accumulated during search, $\mathbf{a}_p \in \{0,1\}^n$ the customer-incidence binary vector of route R_p, and $c_p = D_{\mathrm{T}}(R_p)$ the Euclidean tour length of route R_p. The lexicographic set partitioning problem seeks:*

$$\min_{\mathbf{x}\in\{0,1\}^m} lex\left(\sum_{p=1}^{m} x_p,\ \sum_{p=1}^{m} c_p x_p\right) \quad s.t. \quad \sum_{p=1}^{m} a_{ip} x_p = 1,\ \forall i \in \mathcal{N}. \tag{1}$$

Proposition 1 (Adaptive Certified Scalar Penalty Bound on Finite Route Pool)**.** *Let $\mathcal{P}$ be a finite pool of feasible routes, and let $\mathcal{X}_{SP}(\mathcal{P}) = \{\mathbf{x} \in \{0,1\}^{|\mathcal{P}|} : \sum_{p=1}^{|\mathcal{P}|} a_{ip} x_p = 1,\ \forall i \in \mathcal{N}\}$ denote the set of valid partitioning solutions. Let $c_{\max}^{route} = \max_{p\in\mathcal{P}} c_p$ be the longest single route admitted to $\mathcal{P}$, $\bar{c} = \frac{1}{|\mathcal{P}|}\sum_{p\in\mathcal{P}} c_p$ the pool average route distance, and $K_{\max}$ the model's vehicle ceiling, or the incumbent's fleet size when no ceiling is set. Define:*

$$C_{\max}^{\mathcal{P}} = \max_{\mathbf{x}\in\mathcal{X}_{SP}(\mathcal{P})} \sum_{p=1}^{|\mathcal{P}|} c_p x_p \le K_{\max} \cdot c_{\max}^{route}, \qquad C_{\min}^{\mathcal{P}} = \min_{\mathbf{x}\in\mathcal{X}_{SP}(\mathcal{P})} \sum_{p=1}^{|\mathcal{P}|} c_p x_p. \tag{2}$$

To enforce certified lexicographic fleet priority with guaranteed clearance margin $\epsilon_{margin} > 0$, the adaptive vehicle penalty coefficient is set to:

$$\lambda_{pen}^{cert} = \max\left(\lambda_{pen}^{grid} \cdot \bar{c},\ (1+\epsilon_{margin}) \cdot K_{\max} \cdot c_{\max}^{route}\right), \tag{3}$$

where $\epsilon_{margin} = 0.20$ guarantees $\ge 20\%$ clearance above the theoretical distance gap. For any penalty $\Lambda_{pen} \ge \lambda_{pen}^{cert}$, any optimal solution to the single-objective penalized formulation:

$$\mathbf{x}^* \in \arg\min_{\mathbf{x}\in\mathcal{X}_{SP}(\mathcal{P})} \left(\Lambda_{pen} \sum_{p=1}^{|\mathcal{P}|} x_p + \sum_{p=1}^{|\mathcal{P}|} c_p x_p\right) \tag{4}$$

is guaranteed to attain the minimal vehicle fleet count over $\mathcal{X}_{SP}(\mathcal{P})$, matching the exact lexicographic hierarchy.

Proof. Let $\mathbf{x}_1, \mathbf{x}_2 \in \mathcal{X}_{\mathrm{SP}}(\mathcal{P})$ be two feasible partitions such that $\sum_p x_{1,p} < \sum_p x_{2,p}$. Since vehicle counts are strictly integer-valued, we have $\sum_p x_{2,p} - \sum_p x_{1,p} \ge 1$. The difference in penalized objective values between

$\mathbf{x}_2$ and $\mathbf{x}_1$ is:

$$F(\mathbf{x}_2) - F(\mathbf{x}_1) = \Lambda_{\text{pen}} \left(\sum_p x_{2,p} - \sum_p x_{1,p} \right) + \left(\sum_p c_p x_{2,p} - \sum_p c_p x_{1,p} \right) \tag{5}$$

$$\geq \Lambda_{\text{pen}} \cdot 1 - \left(\sum_p c_p x_{1,p} - \sum_p c_p x_{2,p} \right) \tag{6}$$

$$\geq \Lambda_{\text{pen}} - (C^{\mathcal{P}}_{\max} - C^{\mathcal{P}}_{\min}) \tag{7}$$

$$\geq \Lambda_{\text{pen}} - K_{\max} \cdot c^{\text{route}}_{\max}. \tag{8}$$

Because $\Lambda_{\text{pen}} \geq \lambda^{\text{cert}}_{\text{pen}} \geq (1 + \epsilon_{\text{margin}}) K_{\max} \cdot c^{\text{route}}_{\max} > K_{\max} \cdot c^{\text{route}}_{\max}$, we obtain:

$$F(\mathbf{x}_2) - F(\mathbf{x}_1) \geq \epsilon_{\text{margin}} \cdot K_{\max} \cdot c^{\text{route}}_{\max} > 0, \tag{9}$$

meaning $F(\mathbf{x}_1) < F(\mathbf{x}_2)$. Hence no partition with a larger vehicle count can achieve an objective value lower than or equal to a partition with a strictly smaller vehicle count over $\mathcal{P}$. □

Scope. The proposition concerns an optimal solution of the model over the columns that enter it. When no vehicle ceiling is set, the bound on $C^{\mathcal{P}}_{\max}$ is applied to partitions with at most $K_{\max}$ routes, which is enough for the conclusion provided such a partition exists in the model. The solver places at most 800 pool columns in one model, stops each query at a time limit, and tries four smaller penalty scales first; those smaller scales need not satisfy the bound. A partition returned at a time limit need not be optimal. The solver uses this penalty grid and does not solve a two-stage formulation. The proposition therefore gives no guarantee about the fleet size reached by the search as a whole. Lexicographic order of the reported solution rests on the separate incumbent update rule stated in the main text.

2 Bellman Contraction and the Scope of Convergence Claims for the Entropy-Gated DDQN

Theorem 1 (Bellman Contraction under Entropy Confidence Gating)**.** *Consider a finite Markov decision process with discount $\gamma < 1$, and let the behavior policy select actions from scores that blend the Q-values with prior and bandit terms via the normalized Shannon entropy weight $w_{conf}(\mathbf{s}) \in [0, 1]$. The Bellman optimality operator $\mathcal{T}$ is a γ-contraction in the L_∞-norm with unique fixed point Q^*, and it does not depend on the behavior policy, the replay scheme or the target-network update.*

Proof. The Bellman optimality operator on action-value functions is defined as:

$$(\mathcal{T}Q)(s, a) = R(s, a) + \gamma \mathbb{E}_{s' \sim \mathcal{P}} \left[\max_{a'} Q(s', a') \right]. \tag{10}$$

In Double DQN, target values are evaluated via decoupled argmax selection:

$$y_t = R_t + \gamma Q(s_{t+1}, \arg\max_{a'} Q(s_{t+1}, a'; \theta); \theta^-). \tag{11}$$

By the standard non-expansion property of max, for any value functions Q_1, Q_2:

$$\left| \max_a Q_1(s, a) - \max_a Q_2(s, a) \right| \leq \max_a |Q_1(s, a) - Q_2(s, a)| \leq \|Q_1 - Q_2\|_\infty, \tag{12}$$

which directly implies:

$$\|\mathcal{T}Q_1 - \mathcal{T}Q_2\|_\infty \leq \gamma \|Q_1 - Q_2\|_\infty. \tag{13}$$

Because the discount factors satisfy $\gamma < 1$ (0.97 for the Micro controller and 0.95 for the Macro controller), $\mathcal{T}$ is a strict contraction mapping on the complete metric space $(\mathbb{R}^{|\mathcal{S}| \times |\mathcal{A}|}, \| \cdot \|_\infty)$.

The entropy gating coefficient $w_{\text{conf}}(\mathbf{s})$ affects only the behavior policy during data collection. After a warmup that samples from the bandit, the implementation explores with probability ε by sampling from the prior and bandit, and otherwise takes $\arg\max_a Q_{\text{eff}}(\mathbf{s}, a)$. Because the target in Double DQN does not use Q_{eff}, the resulting transitions affect which parts of $\mathcal{T}$ are sampled but not $\mathcal{T}$ itself, so the gate leaves the fixed point of $\mathcal{T}$ unchanged.

Scope. The contraction property establishes only that Q^* is well defined for a finite MDP; it is not a convergence result for the solver. Convergence of tabular Q-learning to Q^* additionally requires a Markov state, infinitely frequent visits of every state-action pair and suitable step sizes. None of these conditions is established here: the controllers observe aggregated features that are not a sufficient statistic of the search state (main text, macro controller section), they use nonlinear function approximation with replay and target networks, and each run is finite. We therefore claim no convergence of $Q(\cdot;\theta)$ to Q^*, and the learned controllers are evaluated empirically. □

3 Worst-Case Work of the Implemented Ejection Chain

Proposition 2 (Polynomial Worst-Case Work)**.** *Let R_{target} be the route selected for elimination, with m customers. Let K be the number of remaining routes and L the maximum route length reached during the procedure. The implemented procedure performs $O(mK^3L^4)$ elementary operations in the worst case, and $O(mK^2L^3)$ when Level 3 is not run. It terminates, and any plan it returns is feasible with one vehicle fewer.*

Proof. The procedure processes the m customers once each, by decreasing demand. For one customer, finding the best insertion into one route tries at most $L+1$ positions and replays the modified route for each, at $O(L)$ cost, for a total of $I = O(L^2)$. It does not use cached slack queries.

Level 1 evaluates the best insertion into each of K routes, costing $O(KI) = O(KL^2)$. Level 2 considers each of at most KL customers v for ejection. For each, it evaluates I for u in the reduced route and then at most K routes for v, costing $O(KL \cdot KI) = O(K^2L^3)$. Level 3 runs only when at most 30 routes remain. For each of at most KL choices of v, it considers at most KL choices of w, and for each pair evaluates at most K routes for w, costing $O(KL \cdot KL \cdot KI) = O(K^3L^4)$. Summing over m customers gives the bound.

All loops are finite, and each customer is processed once, so the procedure terminates. The first customer that cannot be placed ends it without a result. A returned plan is constructed from replayed feasible routes, and the solver validates its feasibility and fleet size before returning it. □

The bound is not small, and the procedure has no candidate beam and no internal time check. The solver checks its deadline after the procedure returns, and the supervisor enforces the cutoff independently. Search progress within one call is therefore not interruptible by the solver.

4 Derivation of Savelsbergh Forward Time Slack Insertion Delay Condition

Proposition 3 (Forward Time Slack Insertion Feasibility)**.** *Let a route visit sequence be $(v_0, v_1, \ldots, v_m, v_{m+1})$. For each node v_k, let e_k and l_k denote the earliest and latest service start times, s_k the service duration, t_k the arrival time, and $B_k = \max(t_k, e_k)$ the service begin time. The forward time slack at node v_k is defined as:*

$$F_k = \min_{k \le j \le m+1} \left\{ l_j - B_j + \sum_{p=k+1}^{j} W_p \right\}, \tag{14}$$

where $W_p = B_p - (B_{p-1} + s_{p-1} + d_{p-1,p})$ is the waiting time at node v_p.

When inserting a new customer u between v_i and v_{i+1}, let Δt_u denote the arrival delay shifted to v_{i+1}, given by:

$$\begin{aligned} \Delta t_u &= B_u + s_u + d_{u,i+1} - (B_i + s_i + d_{i,i+1}) \\ &= \max\big(0,\ e_u - (B_i + s_i + d_{i,u})\big) + d_{i,u} + s_u + d_{u,i+1} - d_{i,i+1}. \end{aligned} \tag{15}$$

Assume the route up to v_i is unchanged and u itself is served by l_u. The modified route remains time-window feasible for all downstream nodes $v_{i+1}, \ldots, v_{m+1}$ if and only if:

$$\max\big(0,\ \Delta t_u - W_{i+1}\big) \le F_{i+1}. \tag{16}$$

In particular, $\Delta t_u \le F_{i+1}$ is sufficient, and it is also necessary when $W_{i+1} = 0$.

Proof. Let B'_j and W'_j denote the updated service start times and waiting times for downstream nodes $j \ge i{+}1$ following the insertion of customer u. At node v_{i+1}, the arrival time increases by Δt_u. If $\Delta t_u \le W_{i+1}$, the arrival delay is entirely absorbed by the existing waiting time at v_{i+1}, resulting in $B'_{i+1} = B_{i+1}$ and zero propagated delay to subsequent nodes $j > i + 1$.

In general, the shift in service start at v_{i+1} is $\Delta B_{i+1} = \max(0, \Delta t_u - W_{i+1})$, and waiting at later nodes absorbs it in turn:

$$\Delta B_j = \max\Big(0,\ \Delta B_{i+1} - \sum_{p=i+2}^{j} W_p\Big), \quad j \ge i + 1. \tag{17}$$

Downstream feasibility requires $B_j + \Delta B_j \le l_j$ for every $j \in \{i + 1, \ldots, m + 1\}$. Since the original route is feasible, $l_j - B_j \ge 0$, so a zero shift always satisfies the condition, and for a positive shift it is equivalent to:

$$\Delta B_{i+1} \le l_j - B_j + \sum_{p=i+2}^{j} W_p. \tag{18}$$

Taking the minimum over j gives $\Delta B_{i+1} \le F_{i+1}$. With F and W cached, this check costs $O(1)$ per query. Our implementation stores the equivalent latest feasible arrival time at each position, which is computed from the suffix of the route. □

5 Algorithm Specification

Algorithm 1 follows the implementation used in the evaluation. Stagnation-triggered pool seeding, reheating and other minor rules are in the published code and are omitted here.

6 Reward Formulation

Table S1 lists the coefficients of the macro segment reward $R_{\text{seg}} = 0.30[\text{Base} + \lambda_{\text{fleet}}\Psi_{\text{NV}} + (1 - \lambda_{\text{fleet}})\Psi_{\text{TD}} + \text{Bonus}_{\text{NV}}] + \Delta\Phi$. These constants were set by hand during development to balance the scales of discrete vehicle eliminations and continuous distance reductions. We did not retain a calibration log, so we cannot document how they were chosen or show that they are independent of earlier runs on the benchmark instances. They were frozen before the sweeps reported in the paper. We did not run a sensitivity analysis on them under the evaluation protocol of the paper.

7 Reference Values

Best-known fleet sizes and distances come from the SINTEF VRPTW benchmark portal and are stored in the published repository. They are used for descriptive gaps only and for no inference. In the first sweep, the hybrid also read the fleet sizes during search, as disclosed in the main text. Best-known values are heuristic records, not proven optima in general, and they may change over time.

8 Per-Instance Results

[PENDING: generate from the rerun records: for each of the 74 instances at 300 s, mean fleet and matched-fleet distance over five seeds for each comparator, with availability counts, and the same at 10, 60 and 900 s where completed. Generate family variability tables from the same records. Include instances where the proposed configuration loses.]

Algorithm 1 Tri-Level Hybrid DDQN-ALNS for VRPTW

Require: Instance $\mathcal{I}$, Hyperparameters Θ, Time budget T_{budget}
Ensure: Best validated plan s^*

$s_{\text{cur}} \leftarrow$ BuildGreedyInitialSolution($\mathcal{I}$, heatmap)
if the Guided Ejection Search phase is on **then**
    $s_{\text{cur}} \leftarrow \text{GES}(s_{\text{cur}})$ for 10% of the remaining budget
end if
$s^* \leftarrow s_{\text{cur}}$; $\mathcal{P} \leftarrow \text{ExtractRoutes}(s_{\text{cur}})$; $T \leftarrow 0.05\, C(s_{\text{cur}})/\ln 2$, with instance scaling
Initialize Q_{macro}, Q_{micro}, ϕ_{LAC} with random weights; initialize bandits, UCB statistics and penalty multiplier λ
while less than 80% of T_{budget} has elapsed **do**
    if the first phase is active and its end condition holds **then**
        End the first phase; $s_{\text{cur}} \leftarrow s^*$
    end if
    $a_k \leftarrow \pi_{\text{macro}}(\mathbf{s}_k^{\text{macro}})$ if stagnation is at least 12 iterations, else DEFAULT
    Map a_6 to DEFAULT after the first phase; $T \leftarrow T \cdot \text{temp_boost}(a_k)$
    for each iteration of the segment, while time remains **do**
        Select (d, r): bandit during warmup, then ε-greedy over $Q_{\text{eff}}(\mathbf{s}_t^{\text{micro}}, \cdot)$
        $s_{\text{cand}} \leftarrow \text{Repair}(\text{Destroy}(s_{\text{cur}}, d), r)$
        if $a_k = a_5$, or a route has at most 4 customers and $t \bmod 4 = 0$, or stagnation is a positive multiple of 20 **then**
            Replace s_{cand} by the GEC result if it uses fewer vehicles
        end if
        if a_k allows recombination, none ran in this segment, and the plateau and pool conditions hold **then**
            Refine s_{cand} with Set Partitioning over $\mathcal{P}$ via HiGHS
        end if
        if the first phase is active or $a_k \in \{a_5, a_6\}$ **then**
            Decide by penalized Simulated Annealing on C_{pen}
        else if s_{cand} is feasible, keeps the fleet, and increases distance **then**
            Decide by $(p_t \geq \tau_{\text{acc}}(t)) \vee (P_{\text{SA}} > U)$; after warmup, $p_t = \phi_{\text{LAC}}(\mathbf{z}_t)$
        else
            Reject infeasible candidates and disallowed fleet increases; accept others by the rules in the main text
        end if
        if s_{cand} is accepted **then**
            Optionally apply local search under the time budget; add routes to $\mathcal{P}$; $s_{\text{cur}} \leftarrow s_{\text{cand}}$
            if s_{cand} is feasible and $f(s_{\text{cand}}) \prec f(s^*)$ **then**
                $s^* \leftarrow s_{\text{cand}}$; submit s^* for certification
            end if
        end if
        $T \leftarrow 0.99975 \cdot \text{temp_decay_scale}(a_k) \cdot T$
        Compute R_t; store the Micro transition; update Q_{micro}, UCB statistics and ϕ_{LAC}
    end for
    Compute R_{seg}; store the Macro transition when the controller acted; update Q_{macro}
end while
Tail phases until T_{budget}: recombination; fixed-fleet polish; dense column generation; route elimination; distance polish; final recombination
return s^*

Table S1: Coefficients of the Macro Segment Reward and the Micro Iteration Reward.

Component	Term	Macro	Micro	Role
Fleet Term Ψ_{NV}	$\Delta N_{\mathrm{V,best}}$	8.0	3.0	Vehicles removed from the best solution
	ΔC_{best}	1.2	0.40	Percent distance gain of the best solution, floored at zero
	$\Delta N_{\mathrm{V,cur}}$	5.0	2.0	Vehicles removed from the current solution
	ΔC_{cur}	0.6	0.20	Percent distance gain of the current solution, floored at zero
Distance Term Ψ_{TD}	$\Delta N_{\mathrm{V,best}}$	3.0	0.50	As above
	ΔC_{best}	3.5	1.80	As above
	$\Delta N_{\mathrm{V,cur}}$	2.0	0.30	As above
	ΔC_{cur}	1.8	0.90	As above
Base	Constant, or accepted move	-0.20	$+0.05$	Segment step cost; base of an accepted Micro move
	Rejected move	—	-0.08	Micro base when the candidate is rejected
	Local-search pass	-0.04	—	Per pass set by the mode
	Recombination mode	-0.06	—	Modes that may recombine
	Low acceptance	-0.15	—	At most 10% of moves accepted in the segment
Fleet Changes	Per vehicle removed, best	15.0	15.0	Bonus
	Per vehicle removed, current	5.0	5.0	Bonus
	Per vehicle added, current	—	-0.5	Accepted fleet increase
Overall	Scale on the bracket	0.30	0.45	The shaping term $\Delta\Phi$ is added after scaling
	Discount in $\Delta\Phi$	0.95	0.97	Same as the controller's discount